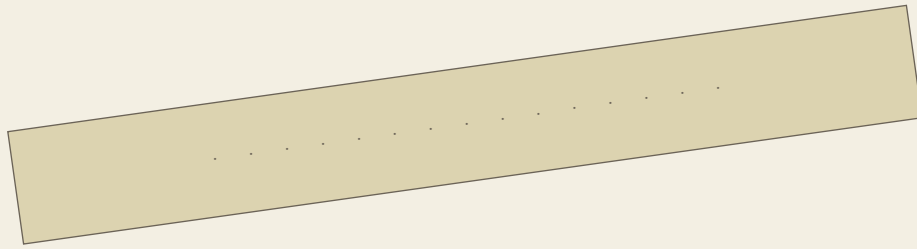


THE LEGACY OF ALBERTA

DE VOLLEDIGE OPLOSSINGEN

Deel 2 – alles verklapt, achter het zegel

een gids in twee delen
voor de zolder van grootmoeder Alberta



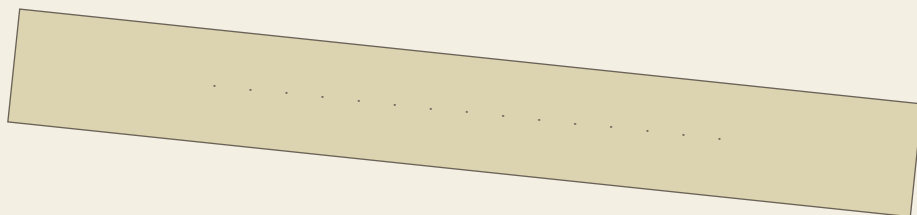
**VERBREEK HET
ZEGEL NIET,
TENZIJ...**



Achter deze pagina staan de **volledige oplossingen**: elke herstellende regel code, elk trace-antwoord, en de weg naar alle vier de eindjes van *Seven Little Goats*.

Eén repository, één zolder. Wie hier te vroeg gluurde, bedriegt alleen zichzelf. Vastgelopen én heeft **?** in het spel geen soelaas gebracht, én heeft deel 1 je niet verder geholpen? Pas dan knip je langs de stippellijn.

(Er is geen echte stippellijn. Dat is het punt.)



JE HEBT HET ZEGEL VERBROKEN

Goed. Dan zit je écht vast, of je hebt gewoon gewonnen en wil nakijken. Beide mag. Hieronder staat alles: de exacte code die je hoort te herstellen, elk trace-antwoord, en de volledige weg naar alle vier de eindes van *Seven Little Goats*. Geen duwtjes meer – antwoorden.

Elke code-oplossing hieronder is letterlijk het fragment uit Alberta's broncode in `seven-little-goats/src/`. Ze compileert en ze draait; dat is de hele bedoeling. Tik ze niet klakkeloos over als je nog kan leren – maar als je hier bent, weet je dat zelf het best.

Een enkele puzzel toont je een willekeurige variant (het spel kiest er een op basis van je seed). Waar dat speelt, staan beide varianten hieronder, met de fout en de fix.

LEVEL 1 – KLASSE EN INSTANTIE: ZEVEN UIT ÉÉN VORM

» Puzzel 1 – herstel de constructor van `Voorwerp`

De constructor moet elk veld van de verse doos vullen met de vorm

```
this.<veld> = <parameter>;
```

► **Variant A** toont `naam = naam;`. De verwijzing naar de doos zelf mist. Fix:

```
this.naam = naam;
```

► **Variant B** toont `beschrijving = this.beschrijving;`. De toewijzing staat omgekeerd. Fix: `this.beschrijving = beschrijving;`

De volledige, correcte klasse:

```
class Voorwerp {

    private String naam;
    private String beschrijving;
    private int kracht;

    Voorwerp(String naam, String beschrijving) {
        this.naam = naam;
        this.beschrijving = beschrijving;
        this.kracht = 0;
    }

    Voorwerp(String naam, String beschrijving, int kracht) {
        this.naam = naam;
        this.beschrijving = beschrijving;
        this.kracht = kracht;
    }

    String getNaam() {
        return naam;
    }

    int getKracht() {
        return kracht;
    }

}
```

» Puzzel 2 – schrijf Geitje van nul

```
class Geitje {  
  
    private String naam;  
    private Schuilplaats schuilplaats;  
    private boolean gered;  
  
    Geitje(String naam, Schuilplaats schuilplaats) {  
        this.naam = naam;  
        this.schuilplaats = schuilplaats;  
        this.gered = false;  
    }  
  
    String getNaam() {  
        return naam;  
    }  
  
    Schuilplaats getSchuilplaats() {  
        return schuilplaats;  
    }  
}
```

» Puzzel 3 – verklaar: klasse versus instantie

Alberta's model: "Een klasse is de blauwdruk – het plan dat je één keer tekent; een instantie is één doos die je naar dat plan bouwt, met eigen waarden in de velden." Komt jouw zin in de kern overeen? Typ **juist**.

LEVEL 2 – SIGNATUREN: WAT ERIN GAAT, WAT ERUIT KOMT

» Puzzel 1 – Parsons: de zoekmethode

De afleider `return gezochteNaam;` laat je liggen (die geeft de parameter terug in plaats van het gevonden object). De juiste volgorde:

```
Voorwerp zoek(String gezochteNaam) {  
    for (int i = 0; i < inventaris.size(); i++) {  
        Voorwerp huidig = inventaris.get(i);  
        if (huidig.getNaam().equals(gezochteNaam)) {  
            return huidig;  
        }  
    }  
    return null;  
}
```

» Puzzel 2 – herstel de signaturen van Speler

- **Variant A** toont `void getLevenspunten()`. Een getter geeft iets terug. Fix:
`int getLevenspunten()`
- **Variant B** toont `void setLevenspunten()`. De trechter mist. Fix:
`void setLevenspunten(int nieuweWaarde)`

De volledige, correcte klasse:

```
class Speler {

    private final int MAX_LEVENSPUNTEN = 20;
    private int levenspunten;
    private ArrayList<Voorwerp> inventaris;

    int getLevenspunten() {
        return levenspunten;
    }

    void setLevenspunten(int nieuweWaarde) {
        if (nieuweWaarde < 0) {
            nieuweWaarde = 0;
        }
        if (nieuweWaarde > MAX_LEVENSPUNTEN) {
            nieuweWaarde = MAX_LEVENSPUNTEN;
        }
        this.levenspunten = nieuweWaarde;
    }

    Voorwerp zoek(String gezochteNaam) {
        for (int i = 0; i < inventaris.size(); i++) {
            Voorwerp huidig = inventaris.get(i);
            if (huidig.getNaam().equals(gezochteNaam)) {
                return huidig;
            }
        }
        return null;
    }

    boolean verwijder(String teVerwijderenNaam) {
        for (int i = 0; i < inventaris.size(); i++) {
            if (inventaris.get(i).getNaam().equals(teVerwijderenNaam)) {
                inventaris.remove(i);
                return true;
            }
        }
        return false;
    }
}
```

» Puzzel 3 – trace: parameter schaduwat attribuut

De eerste regel drukt de **parameter** af, de tweede **this.levenspunten** (het **attribuut**, vast op 20). Het antwoord hangt af van de meegegeven waarde:

- **toon(3)** → 3 20
- **toon(5)** → 5 20
- **toon(7)** → 7 20

LEVEL 3 – VOORWAARDEN: DE DEUR OP SLOT

» Puzzel 1 – herstel de klem in **setLevenspunten**

- **Variant A** toont **if (nieuweWaarde > 0)**. De ondergrens staat de verkeerde kant op. Fix: **if (nieuweWaarde < 0)**

- **Variant B** mist het hele bovengrens-blok. Voeg het terug toe (zie hieronder).

De volledige, correcte methode:

```
void setLevenspunten(int nieuweWaarde) {
    if (nieuweWaarde < 0) {
        nieuweWaarde = 0;
    }
    if (nieuweWaarde > MAX_LEVENSPUNTEN) {
        nieuweWaarde = MAX_LEVENSPUNTEN;
    }
    this.levenspunten = nieuweWaarde;
}
```

» **Puzzel 2 – vind de fout: && versus ||**

De fout zit op **regel 1**. De code gebruikt `||` (of) waar `&&` (en) hoort: “de wolf verslagen is EN je de sleutel hebt” vraagt dat beide waar zijn. Met `||` gaat de poort al open bij alleen de sleutel. Typ **regel 1**, of `&&`, of “beide moeten waar zijn”.

» **Puzzel 3 – trace: de validatie-cascade**

De eerste tak die klopt wint, en de randen tellen: `<= 0` pakt ook 0, `< 10` laat 10 net vallen naar de laatste tak.

- `lp = 0` → verslagen
- `lp = 5` → gewond
- `lp = 10` → gezond

LEVEL 4 – REFERENTIES: TWEE PIJLEN, ÉÉN DOOS

» **Puzzel 1 – herstel de buur-bedrading (verbindKamers)**

Beide varianten laten één richting-setter overal vervangen door een verkeerde, zodat een terugweg nooit meer wordt gelegd.

- **Variant A** heeft overal `setNoord` waar `setZuid` hoort. Herstel de twee `setZuid`-aanroepen (in de noord-tak en de zuid-tak).
- **Variant B** heeft overal `setOost` waar `setWest` hoort. Herstel de twee `setWest`-aanroepen (in de oost-tak en de west-tak).

De volledige, correcte methode:

```
private void verbindKamers(Kamer eerste, String richting, Kamer tweede) {
    if (richting.equals("noord")) {
        eerste.setNoord(tweede);
        tweede.setZuid(eerste);
    } else if (richting.equals("oost")) {
        eerste.setOost(tweede);
        tweede.setWest(eerste);
    } else if (richting.equals("zuid")) {
        eerste.setZuid(tweede);
        tweede.setNoord(eerste);
    } else if (richting.equals("west")) {
        eerste.setWest(tweede);
    }
}
```

```

        tweede.setOost(eerste);
    }
}

```

» Puzzel 2 – trace: aliasing

`eerste` en `tweede` wijzen naar dezelfde kamer. Wat je via `tweede.setNoord(...)` zet, lees je via `eerste.getNoord().getNaam()` weer uit: het is precies de naam van de doel-kamer.

- doel `Dorpsplein` → `Dorpsplein`
- doel `Bospad` → `Bospad`
- doel `Rivieroever` → `Rivieroever`

» Puzzel 3 – verklaar: wat betekent `null`?

Alberta's model: "null is een pijl die naar geen enkele doos wijst: er is in die richting geen buurkamer, dus geen uitgang." Komt jouw zin in de kern overeen? Typ `juist`.

LEVEL 5 – LUSPATRONEN: GEITJE VOOR GEITJE

» Puzzel 1 – schrijf de twee lus-methoden

`telWapens` is de tel-kaart (teller op 0, hoog op bij een treffer);

`sterksteVoorwerp` is de uiterste-kaart (begin op `null`, onthoud de sterkste).

```

int telWapens() {
    int aantal = 0;
    for (int i = 0; i < inventaris.size(); i++) {
        if (inventaris.get(i).getKracht() > 0) {
            aantal++;
        }
    }
    return aantal;
}

Voorwerp sterksteVoorwerp() {
    Voorwerp sterkste = null;
    for (int i = 0; i < inventaris.size(); i++) {
        Voorwerp huidig = inventaris.get(i);
        if (sterkste == null || huidig.getKracht() > sterkste.getKracht()) {
            sterkste = huidig;
        }
    }
    return sterkste;
}

```

» Puzzel 2 – Parsons: de string-builder

De afleider `regel = hier.get(i).getNaam();` overschrijft de regel in plaats van eraan toe te voegen – die laat je liggen. De juiste volgorde:

```

String regel = "Je kan hier meenemen: ";
for (int i = 0; i < hier.size(); i++) {

```

```
    if (i > 0) {  
        regel = regel + ", ";  
    }  
    regel = regel + hier.get(i).getNaam();  
}  
System.out.println(regel);
```

» Puzzel 3 – welke patroonkaart?

Optie 2 – totaliseren. Bij `som = som + schadeLog[i]` groeit `som` met de waarde zelf, niet met één per element. Tellen zou `aantal++` zijn; dit is de totaliseer-kaart.

LEVEL 6 – INDEX EN OFF-BY-ONE: DE LAATSTE PLANK

» Puzzel 1 – herstel de verwijder-lus in `verwijderVoorwerp`

► **Variant A** toont `i <= voorwerpen.size()`. Eén plank te ver. Fix:

```
i < voorwerpen.size()
```

► **Variant B** gebruikt een `while` waar een `for` de nette keuze is. Herschrijf naar de `for`-lus hieronder.

De volledige, correcte methode:

```
void verwijderVoorwerp(String teVerwijderenNaam) {  
    for (int i = 0; i < voorwerpen.size(); i++) {  
        if (voorwerpen.get(i).getNaam().equals(teVerwijderenNaam)) {  
            voorwerpen.remove(i);  
            return;  
        }  
    }  
}
```

» Puzzel 2 – trace: de laatste afgedrukte index

De lus loopt van 0 tot `size()` (strikt kleiner), dus de laatste index is `size()` min één.

- 3 voorwerpen → 2
- 5 voorwerpen → 4
- 7 voorwerpen → 6

» Puzzel 3 – vind de fout: de rondelus

De fout zit op **regel 1**. `ronde <= 5` speelt zes rondes (0 t/m 5) waar er vijf horen. Met `ronde < 5` blijf je netjes bij vijf. Typ **regel 1**, of `<=`, of “één te ver”.

LEVEL 7 – ZOEKEN EN DE DUBBELE PIJL: WAAR HET JONGSTE ZIT

» Puzzel 1 – schrijf de zoeklus (zoekGeitje)

```
Geitje zoekGeitje(String gezochteNaam) {
    for (int i = 0; i < geitjes.size(); i++) {
        Geitje huidig = geitjes.get(i);
        if (huidig.getNaam().equals(gezochteNaam)) {
            return huidig;
        }
    }
    return null;
}
```

» Puzzel 2 – herstel de null-veilige keten (toonSchuilplaatsen)

- **Variant A** volgt `geitje.getSchuilplaats().getKamer().getNaam()` zonder null-controle: een nog-opgeslokt geitje heeft geen schuilplaats en de keten loopt op niets.
- **Variant B** controleert wel op null, maar mist de schakel `getKamer()` (en toont alleen de schuilplaatsnaam).

De volledige, correcte methode:

```
private void toonSchuilplaatsen() {
    System.out.println("Het jongste geitje vertelt waar elk voortaan schuilt:");
    for (int i = 0; i < geitjes.size(); i++) {
        Geitje geitje = geitjes.get(i);
        Schuilplaats schuilplaats = geitje.getSchuilplaats();
        if (schuilplaats == null) {
            System.out.println("- " + geitje.getNaam() + ": nog niet gevonden");
        } else {
            System.out.println("- " + geitje.getNaam() + ": " +
                schuilplaats.getNaam()
                + " (" + schuilplaats.getKamer().getNaam() + ")");
        }
    }
}
```

» Puzzel 3 – trace: de geketende getter

Controleer eerst of de schuilplaats `null` is. Het jongste geitje zit in de klokkast (schuilplaats → Geitenhuisje); de broer is nog opgeslokt (schuilplaats `null`).

- aangeroepen op `jongste` → `Geitenhuisje`
- aangeroepen op `broer` → `nog niet gevonden`

SEVEN LITTLE GOATS: DE VIER EINDES, STAP VOOR STAP

Na level 7 speel je Alberta's spel uit als Roodkapje. Hieronder staan de vier eindjes, elk als een reeks commando's die je letterlijk kan intypen. Ze zijn deterministisch: het aanvalspatroon van de wolf ligt vast, dus dezelfde

commando's leiden altijd tot hetzelfde einde. (Dit zijn exact de geverifieerde test-scripts uit `seven-little-goats/test-scripts/`.)

» De kaart

Alles ligt op één zuid-lijn, met het dorpsplein als spil:

```
[Geitenhuisje]   (klokkast: het jongste geitje)
  | zuid
[Dorpsplein]     (twee koeken op de marktkraam)
  |--oost--[Molen]       (bloem, kruik melk; jachthond)
  |--west--[Kruidenier]  (krijt)
  | zuid
[Bospad]
  | zuid
[Oude eik]       (raaf: geef koek -> gladde kiezels)
  | zuid
[Grootmoeders huisje] (zilveren schaar)
  | zuid
[Wolvenspoor]
  | zuid
[Rivieroever]    (losse stenen; de showdown met de jonge wolf)
```

» De vechtrekenkunde

Je basiskracht is 2. Met het `keukenmes` erbij deel je **5 schade** per `val aan` uit. De jonge wolf heeft 18 levenspunten en smeekt zodra hij op 5 of minder zakt. Drie keer `val aan` met het mes ($3 \times 5 = 15$) brengt hem van 18 naar 3 – en dan smeekt hij. Op dat smeekmoment kies je: `maak af` of `spaar`.

De `rode mantel` vangt elke ronde 1 schade op; onmisbaar om de vijf rondes heel door te komen. Zonder mes én zonder mantel overleef je het wolfgevecht niet – dat is het game over hieronder.

» Einde 1 – De schaar en de stenen (het beste einde)

De wolf verslagen mét grootmoeders `zilveren schaar` én iets steenachtigs (de `gladde kiezels` van de raaf, of de losse `stenen` aan de oever). De zes geitjes worden bevrijd, de buik met stenen gevuld, en het jongste geitje vertelt waar elk broertje voortaan schuilt (de null-veilige endgame-keten uit level 7).

```
pak rode mantel
pak keukenmes
pak mandje
ga zuid
pak koek
pak koek
ga zuid
ga zuid
geef koek
ga zuid
pak zilveren schaar
ga zuid
ga zuid
vecht
val aan
val aan
```

```
val aan  
maak af
```

Wat er gebeurt: je rust je uit in het geitenhuisje (mantel, mes, mandje), pakt op het dorpsplein twee koeken (het **mandje** maakt dat mogelijk), ruilt er eentje bij de raaf op de oude eik voor **gladde kiezels**, neemt de **zilveren schaar** bij grootmoeder mee, en zakt af tot de rivieroever. Drie halen met het mes, de wolf smeekt, **maak af**. Omdat je schaar én kiezels bij je hebt, valt het beste einde. (Je mag de raaf-ruil overslaan en in de plaats aan de oever **pak stenen** doen – beide tellen als “de stenen”.)

» Einde 2 – De afrekening (kille wraak)

De wolf verslagen, maar **zonder** de schaar of zonder de stenen. Hij is dood; de geitjes blijven waar ze zijn. Koud en onaf.

```
pak rode mantel  
pak keukenmes  
ga zuid  
ga zuid  
ga zuid  
ga zuid  
ga zuid  
ga zuid  
vecht  
val aan  
val aan  
val aan  
maak af
```

Zes keer **ga zuid** brengt je rechtstreeks van het geitenhuisje naar de rivieroever (dorpsplein, bospad, oude eik, grootmoeders huisje, wolvenspoor, oever). Je hebt mes en mantel, maar geen schaar en geen stenen. Drie halen, **maak af** – en het wordt de afrekening.

» Einde 3 – De les (genade)

Identiek aan de afrekening, tot het smeekmoment. Daar kies je **spaar** in plaats van **maak af**. De wolf spuwt de geitjes uit en glipt het bos in.

```
pak rode mantel  
pak keukenmes  
ga zuid  
ga zuid  
ga zuid  
ga zuid  
ga zuid  
ga zuid  
vecht  
val aan  
val aan  
val aan  
spaar
```

» Einde 4 – Game over

Ga zonder wapen én zonder mantel het gevecht in, en de wolf wint. Vijf keer **val aan** met kale vuisten (kracht 2) knabbelt maar traag aan zijn 18 levenspunten, terwijl zijn vaste patroon (3, 5, 2, 6, 4 = 20 schade) je 20 levenspunten precies leegtrekt tegen de vijfde ronde.

```
ga zuid
ga zuid
ga zuid
ga zuid
ga zuid
ga zuid
vecht
val aan
val aan
val aan
val aan
val aan
```

EN JE "CIJFER"? ALBERTA'S OORDEEL

Op het einde geeft Alberta een warme, droge terugblik op basis van hoeveel hints je in totaal vroeg – nooit een straf, nooit een buis. Elk van de vier tiers is positief; het verschil is de knipoog.

Tier	Hints totaal	Alberta zegt (kort)
De meesterhand	0-3	"Ik had het niet beter gekund, en dat zeg ik niet snel."
De vakvrouw	4-10	"Nu en dan een blik in de kantlijn, en dan weer door."
De doorzetter	11-20	"Je hebt vaak om hulp gevraagd en telkens opnieuw doorgezet."
Samen geraakt	21+	"We hebben dit samen gedaan, jij en ik en een hoop hints."

Het zegel verbroken en deel 2 gebruikt? Dat telt niet mee in Alberta's som – alleen de **?**-hints in het spel doen dat. Maar je weet het zelf. Eén repository, één zolder. Volgende keer geraak je er zonder.

EN DAN DE DISKETTE

Na het oordeel doet de pc nog één ding: hij schrijft alles wat je hersteld hebt weg naar de diskette die al die tijd in de drive zat. Je ziet ze groot in beeld, met een etiket in Alberta's handschrift – ze had het al geschreven voor een spel dat toen nog niet af was. Enter, en je klikt ze eruit.

Dat is de echte prijs, en hij is geen beeld: wat op die diskette staat, staat ook in **seven-little-goats/**. Open het in IntelliJ, lees het, draai het zelf.

Nu de diskette nog. Ze gaat mee.